

Database Migration — Interview Answers

Questions 27–31: Zero-downtime migration, chunking, rollback, data consistency, and delayed production issues

27. How would you migrate lakhs/millions of active user data from the legacy DB to the new DB without downtime?

I would use a **phased, chunk-based migration** rather than migrating everything at once.

- **1. Prepare the new DB:** Create the schema, indexes, constraints, and make sure the application is compatible.
- **2. Initial bulk migration:** Copy users in manageable chunks, for example 10K–50K records at a time, depending on database performance.
- **3. Validate each chunk:** Compare record counts, checksums/hashes, important fields, and relationships.
- **4. Handle ongoing changes:** Users continue updating data while migration is running. Use CDC/change-data-capture, replication, an outbox/event mechanism, or application-level dual writes to keep the new DB synchronized.
- **5. Gradually move traffic:** Use a feature flag or routing layer to send a small percentage of users to the new DB.
- **6. Monitor:** Track errors, latency, data consistency, and business metrics.
- **7. Increase traffic gradually:** For example, 1% → 5% → 25% → 50% → 100%, depending on stability.
- **8. Keep the legacy DB available:** Retain a fallback path until the migration is proven stable.
- **Key point:** The data migration and traffic migration are separate. The application does not need to be taken offline just because data is being copied.

28. If any kind of issue comes after migration, do we need to go chunk-wise only?

Yes, preferably the **recovery/rollback should be scoped to the affected chunk or users**, rather than rolling back the entire migration.

First identify which users were included in the affected chunk, what data is affected, whether the problem is in the migrated data or the new application, and whether any writes happened after migration.

Possible recovery actions include restoring the affected records from a backup/snapshot, re-copying them from the legacy DB, replaying relevant changes, or temporarily routing affected users back to the legacy system.

I would not automatically roll back everything because millions of already-migrated users may be completely healthy.

- **Important:** The migration process should be **idempotent**, so retrying a failed chunk does not create duplicate or inconsistent data.

29. You are migrating in a chunk and the first chunk causes a production issue. What would you do first?

First, stop the migration/traffic rollout immediately. I would not continue migrating more users while the production issue is not understood.

- **1. Identify impact:** Determine how many users are affected and whether the issue is read-related, write-related, or data-corruption-related.

- **2. Stop further exposure:** If traffic has already moved to the new system, use a feature flag/routing mechanism to send affected users back to the legacy system, if available.
- **3. Preserve evidence:** Capture logs, metrics, request IDs, affected user IDs, migration batch ID, and relevant database state.
- **4. Compare old vs new data:** Determine whether the problem comes from migration logic, schema transformation, missing data, incorrect mapping, application compatibility, or an unrelated application issue.
- **5. Recover:** If the new data is incorrect, restore or re-copy affected records from the trusted legacy source. Fix the issue, test it, and then retry the chunk.
- **Interview line:** “My first action is to stop further exposure and contain the blast radius. I would not continue the migration while production users are being affected.”

30. What could happen in the files/data which are already migrated?

- **1. Data can become stale:** A user may update information in the legacy DB after the record was migrated. Without synchronization, the new DB can become outdated.
- **Example:** Legacy DB has email = old@email.com and New DB has the same value. Later the user changes the legacy value to new@email.com. The new DB now has stale data.
- **2. Transformation bugs:** A field may be converted incorrectly during migration, such as date or format conversions.
- **3. Missing or corrupted data:** Some records or related records may fail to migrate.
- **4. Duplicate records:** A non-idempotent retry can potentially create duplicates.
- **5. Application/schema incompatibility:** The data may exist in the new DB but the new application may interpret it differently.
- **6. New writes can create inconsistencies:** This is one of the biggest concerns in a live migration. The source data continues changing after the initial copy.
- **Therefore:** Monitor data consistency after migration, not just during the migration.

31. What if the issue appears after one day, when the user comes back and finds a problem in their profile?

This is why I would not consider migration successful immediately after the copy finishes. I would keep the **legacy DB as a fallback/source of truth for an appropriate stabilization period**, depending on business requirements.

- **Example flow:** Day 0 migration → Day 1 user uses new system → monitoring/validation → issue detected → compare new DB with legacy DB → identify root cause → repair/replay data or route the user back.

If a user reports that their profile information is incorrect, first compare the user's record between both systems and check the migration/change history.

- **Example:** Legacy DB: User 123 → phone = 9876543210. New DB: User 123 → phone = NULL. Investigate whether the migration missed the field, the transformation was wrong, a post-migration update was lost, or the UI is displaying the wrong value.

For large-scale systems, maintain a **migration audit table** with fields such as user_id, migration_batch_id, migration_status, migrated_at, source checksum/version, target checksum/version, validation_status, and error_reason.

This provides traceability when a user reports an issue several hours or days after migration.

Final interview summary

“I would design the migration so that both migration and recovery are incremental. I would migrate in chunks, validate every chunk, maintain synchronization between the old and new systems while users are active, gradually shift traffic, and keep a rollback/fallback path. If an issue appears immediately or even a day later, I would identify the affected users, compare source and target data, stop further exposure, and repair or replay only the affected data rather than blindly rolling back millions of users.”